



Trabajo Práctico: Introducción a NodeJS

Objetivo:

El objetivo de esta actividad es introducir el desarrollo de servidores utilizando **Node.js**, comprendiendo los conceptos básicos que existen detrás de una API web antes de utilizar frameworks como Express.

A lo largo del trabajo práctico se trabajará con servidores HTTP, rutas, métodos HTTP, respuestas JSON, archivos estáticos, manejo de datos en memoria, lectura del cuerpo de una request y uso de streams.

Este trabajo práctico tiene carácter formativo y de acompañamiento. Su propósito es preparar los conceptos necesarios para el desarrollo posterior de servidores API REST utilizando Node.js y Express.

Consigna general:

Desarrollar una pequeña aplicación backend utilizando **Node.js sin paquetes de terceros**.

No se debe utilizar:

- Express
- Nodemon
- Cors
- Body-parser
- Librerías externas para routing
- Librerías externas para manejo de archivos

Sí se pueden utilizar módulos nativos de Node.js, por ejemplo:

- http
- fs
- path
- url
- crypto
- stream
- perf_hooks



Dominio de trabajo

La aplicación trabajará con una entidad llamada **Character**, que representa personajes épicos.

Cada personaje deberá tener, como mínimo:

```
{  
  id: "1",  
  name: "Arthas",  
  race: "Human",  
  role: "Death Knight",  
  level: 90,  
  universe: "Warcraft"  
}
```

Pueden agregar más propiedades si lo consideran necesario, por ejemplo:

```
{  
  description,  
  weapon,  
  isAlive,  
  imageUrl,  
  power,  
}
```

Requerimientos

1) Crear un servidor HTTP básico con Node.js

Crear un archivo *server.js* que levante un servidor HTTP utilizando el módulo nativo `http`.

El servidor debe escuchar en el puerto 3000.

Al ingresar a:

```
GET /api/characters
```

debe responder con una lista de personajes épicos almacenados en un arreglo en memoria.

Ejemplo de respuesta:

```
[
  {
    "id": "1",
    "name": "Arthas",
    "race": "Human",
    "role": "Death Knight",
    "level": 90,
    "universe": "Warcraft"
  },
  {
    "id": "2",
    "name": "Geralt of Rivia",
    "race": "Human",
    "role": "Witcher",
    "level": 75,
    "universe": "The Witcher"
  }
]
```

También debe existir una ruta simple de prueba:

```
GET /health
```

que responda:

```
{
  "status": "ok"
}
```

2) Servir archivos estáticos desde una carpeta public

Crear una carpeta llamada public con, al menos, los siguientes archivos:

```
public/
  index.html
  styles.css
  app.js
```



El servidor debe poder devolver esos archivos cuando el navegador los solicite.

Por ejemplo:

```
GET /
```

debe devolver public/index.html.

```
GET /styles.css
```

debe devolver public/styles.css.

```
GET /app.js
```

debe devolver public/app.js.

El servidor debe configurar correctamente el header Content-Type según el tipo de archivo:

```
.html → text/html  
.css → text/css  
.js → text/javascript  
.json → application/json
```

Si el archivo solicitado no existe, el servidor debe responder con código 404.

3) Crear una API básica para personajes

Extender el servidor para que permita trabajar con personajes usando diferentes métodos HTTP.

La API debe soportar:

Obtener todos los personajes

```
GET /api/characters
```

Debe devolver todos los personajes.



Obtener un personaje por ID

```
GET /api/characters/:id
```

Ejemplo:

```
GET /api/characters/1
```

Si el personaje existe, debe devolverlo.

Si no existe, debe responder con código 404 y un mensaje adecuado.

Crear un personaje

```
POST /api/characters
```

Debe permitir crear un nuevo personaje enviando datos en formato JSON.

Ejemplo de body:

```
{  
  "name": "Link",  
  "race": "Hylian",  
  "role": "Hero",  
  "level": 80,  
  "universe": "The Legend of Zelda"  
}
```

El servidor debe:

- Leer el body de la request.
 - Convertirlo desde JSON a objeto JavaScript.
 - Crear un nuevo personaje.
 - Asignarle un id.
 - Guardarlo en el arreglo en memoria.
 - Responder con código 201.
 - Devolver el personaje creado.
-



Actualizar un personaje

```
PUT /api/characters/:id
```

Ejemplo:

```
PUT /api/characters/1
```

Debe permitir actualizar un personaje existente.

Si el personaje existe, debe reemplazar o actualizar sus datos y devolver el personaje actualizado.

Si el personaje no existe, debe responder con código 404.

4) Validaciones mínimas

Agregar validaciones básicas para evitar crear personajes incompletos.

Para crear un personaje, como mínimo deben existir:

- name
- race
- role
- level
- universe

Si falta alguno de esos campos, el servidor debe responder con código 400.

Ejemplo:

```
{  
  "error": "Faltan campos obligatorios"  
}
```

También se debe validar que level sea un número.

5) Conectar el HTML con la API

Desde el archivo public/app.js, utilizar fetch para consumir la API creada.



La página index.html debe mostrar en pantalla la lista de personajes obtenidos desde:

```
GET /api/characters
```

Como mínimo, cada personaje debe mostrar:

- Nombre
- Raza
- Rol
- Nivel
- Universo

Opcionalmente, pueden agregar un formulario simple para crear personajes desde el navegador utilizando:

```
POST /api/characters
```

6) Crear un script para copiar archivos usando streams

Crear una carpeta llamada scripts.

Dentro de ella, crear un archivo:

```
scripts/copy-file.js
```

Este script debe copiar un archivo desde una ubicación de origen hacia una ubicación de destino utilizando streams.

No se puede utilizar:

```
fs.copyFile
```

Se debe utilizar:

```
fs.createReadStream  
fs.createWriteStream
```

El script debe recibir el archivo de origen y el archivo destino por argumentos de consola.



Ejemplo de ejecución:

```
node scripts/copy-file.js ./input.txt ./output.txt
```

Al finalizar la copia, debe mostrar por consola cuánto tiempo tardó la operación.

Ejemplo:

```
Archivo copiado correctamente.  
Tiempo total: 18ms
```

También debe manejar errores, por ejemplo:

- Archivo inexistente.
- Ruta inválida.
- Error de lectura.
- Error de escritura.

Buenas prácticas esperadas

El código debe:

- Estar separado en funciones cuando sea necesario.
- Evitar duplicación innecesaria.
- Usar nombres claros para variables y funciones.
- Responder con códigos HTTP adecuados.
- Mantener los datos en memoria durante la ejecución del servidor.
- Ser legible para otro desarrollador.

Evitar

- Usar Express u otros frameworks.
- Usar paquetes externos para leer el body.
- Usar paquetes externos para servir archivos estáticos.
- Responder siempre con código 200, incluso cuando hay errores.
- Devolver HTML cuando la ruta pertenece a la API.
- Duplicar la lógica de respuestas JSON en muchas partes del código.

Estructura de directorios recomendada

```
tp-node-intro/  
  server.js  
  README.md  
  public/  
    index.html  
    styles.css  
    app.js  
  scripts/  
    copy-file.js  
  data/  
    characters.js
```

Opcionalmente, pueden separar más responsabilidades:

```
tp-node-intro/  
  server.js  
  README.md  
  public/  
    index.html  
    styles.css  
    app.js  
  scripts/  
    copy-file.js  
  data/  
    characters.js  
  utils/  
    sendJson.js  
    parseBody.js  
    serveStaticFile.js
```

Modalidad de trabajo

Este trabajo práctico puede realizarse de forma individual o grupal.

No requiere tablero Kanban obligatorio, ya que se trata de una actividad introductoria y formativa. Sin embargo, se recomienda organizar las tareas en pasos pequeños, de manera similar a los trabajos anteriores.

No hay un entregable obligatorio.

Bonus opcional

Algunas ideas posibles para profundizar:

- Agregar DELETE:

```
DELETE /api/characters/:id.
```

- Agregar filtros por query params:

```
GET /api/characters?universe=Warcraft
```

- Agregar búsqueda por nombre:

```
GET /api/characters?name=arthas
```

- Agregar paginación simple:

```
GET /api/characters?page=1&limit=5
```

- Guardar los personajes en un archivo .json.
- Agregar manejo de rutas no encontradas.